Ski Pass Analysis

March 5, 2026

1 Ski Pass Analysis

Ok, the point of this exercise is to take a look at the ski passes available and determine which one to buy for next year. Obviously the benefit to user/business is going to come down to price per day, the number of skiiable days per year, etc…

2 Ski Pass Pricing

Ok, let's first take a look at the ski passes. This was obtained using the scraper in the parent library. Note that the providers use HEAVY bot moderation and these pages change frequently so the scrapers break. I need to just replace it with API calls to Gemini or OpenAI since they collect that info anyway.

```
Notes on Data: These prices do not reflect changes in prices due to pre-purchasing passes or us
```

[1]:
```
import pandas as pd

df = pd.read_csv('/media/kwierman/Data/powderpipeline/ski_passes.csv')

df
```

[1]:
```
   Provider                   Name Age Range  Price
0      Epic        Epic Pass Adult        31   1089
1      Epic        Epic Pass Child      5-12    555
2      Epic         Epic Pass Teen     13-17    869
3      Epic  Epic Pass Young Adult     18-30    869
4      Ikon              Ikon Pass        23   1399
5      Ikon         Ikon Base Pass        23    949
6      Ikon      Ikon Session Pass        23    299
7      Indy          Learn to Turn        23    189
8      Indy         Indy Base Pass        23    368
9      Indy             Indy+ Pass        23    419
10     Indy  Indy Base Add on Pass        23    289
```

Ok, so the prices look pretty comparable. We'll use this later on.

3 Ski Resort Overview

Let's take a look at a map of all the ski resorts and see where they are and which pass they're associated with

[2]:
```
ski_resorts_df = pd.read_csv('/media/kwierman/Data/powderpipeline/ski_resorts.
↪csv')
ski_resorts_df.head()
```

[2]:
```
     Resort_Name Pass_Affiliation  Latitude  Longitude  Base_Elevation_ft  \
0           Vail             Epic   39.6403  -106.3742               8120
1   Breckenridge             Epic   39.4808  -106.0667               9600
2       Keystone             Epic   39.6053  -105.9515               9280
3  Crested Butte             Epic   38.8997  -106.9658               9375
```

```
4   Beaver Creek             Epic   39.6042  -106.5165                8100

   Summit_Elevation_ft
0                11570
1                12998
2                12408
3                12162
4                11440
```

[3]:
```python
import altair as alt
from vega_datasets import data
states = alt.topo_feature(data.us_10m.url, feature='states')

base_map = alt.Chart(states).mark_geoshape(
    fill='lightgray',
    stroke='white'
).project(
    "albersUsa"
).properties(
    title='Ski Resorts in the United States',
    width=600,
    height=400
)

points = alt.Chart(ski_resorts_df).mark_circle().encode(
    longitude='Longitude:Q',
    latitude='Latitude:Q',
    color=alt.Color('Pass_Affiliation:N', scale=alt.Scale(scheme='viridis')), #␣
 ↪Color by the specified column
    size=alt.value(50), # Set a fixed size for better visibility
    tooltip=['Resort_Name', 'Pass_Affiliation'] # Add tooltips for interactivity
)

# Combine the base map and the points
chart = base_map + points
chart.save('../public/ski_resorts.png')

# Display the final chart
chart
```

[3]:
```
alt.LayerChart(…)
```

Ok, so obviously, the scraper didn't get *everything*, which is a bit dissappointing. I'll have to come back to that later. For now, let's focus on the ones that matter.

3.1 Ski Resorts in Washington, Oregon, Alaska and Idaho

I *had* planned on looking at Alaska. I'll have to redo that later. For now, let's take a look at what we've got.

[4]:
```
from geopy.geocoders import Nominatim

geolocator = Nominatim(user_agent="powderpipeline_app")

def get_state_name(row):
    # Format the coordinates
    coordinates = f"{row['Latitude']}, {row['Longitude']}"
    # Use geolocator.reverse() to get location data
    location = geolocator.reverse(coordinates, exactly_one=True)
    # Extract the state name from the raw address dictionary
    if location and 'address' in location.raw:
        # 'state' is the key for state name in the Nominatim response
        return location.raw['address'].get('state', 'Unknown')
    else:
        return 'Unknown'
    # Add a delay to respect API rate limits (crucial for larger dataframes)
    time.sleep(10)

ski_resorts_df['State'] = ski_resorts_df.apply(get_state_name, axis=1)
ski_resorts_df.head()
```

```
---------------------------------------------------------------------------
TimeoutError                              Traceback (most recent call last)
File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:534, in HTTPConnectionPool._make_request(self, conn␣
 ↪method, url, body, headers, retries, timeout, chunked, response_conn,␣
 ↪preload_content, decode_content, enforce_content_length)
    533 try:
--> 534     response = conn.getresponse()
    535 except (BaseSSLError, OSError) as e:

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connection.py:571, in HTTPConnection.getresponse(self)
    570 # Get the response from http.client.HTTPConnection
--> 571 httplib_response = super().getresponse()
    573 try:

File ~/miniconda3/lib/python3.12/http/client.py:1430, in HTTPConnection.
 ↪getresponse(self)
   1429 try:
-> 1430     response.begin()
   1431 except ConnectionError:
```

```
File ~/miniconda3/lib/python3.12/http/client.py:331, in HTTPResponse.begin(self
    330 while True:
--> 331     version, status, reason = self._read_status()
    332     if status != CONTINUE:

File ~/miniconda3/lib/python3.12/http/client.py:292, in HTTPResponse.
 ↪_read_status(self)
    291 def _read_status(self):
--> 292     line = str(self.fp.readline(_MAXLINE + 1), "iso-8859-1")
    293     if len(line) > _MAXLINE:

File ~/miniconda3/lib/python3.12/socket.py:720, in SocketIO.readinto(self, b)
    719 try:
--> 720     return self._sock.recv_into(b)
    721 except timeout:

File ~/miniconda3/lib/python3.12/ssl.py:1251, in SSLSocket.recv_into(self,␣
 ↪buffer, nbytes, flags)
   1248         raise ValueError(
   1249           "non-zero flags not allowed in calls to recv_into() on %s" %
   1250           self.__class__)
-> 1251     return self.read(nbytes, buffer)
   1252 else:

File ~/miniconda3/lib/python3.12/ssl.py:1103, in SSLSocket.read(self, len,␣
 ↪buffer)
   1102 if buffer is not None:
-> 1103     return self._sslobj.read(len, buffer)
   1104 else:

TimeoutError: The read operation timed out

The above exception was the direct cause of the following exception:

ReadTimeoutError                          Traceback (most recent call last)
File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:787, in HTTPConnectionPool.urlopen(self, method,␣
 ↪url, body, headers, retries, redirect, assert_same_host, timeout,␣
 ↪pool_timeout, release_conn, chunked, body_pos, preload_content,␣
 ↪decode_content, **response_kw)
    786 # Make the request on the HTTPConnection object
--> 787 response = self._make_request(
    788     conn,
    789     method,
    790     url,
    791     timeout=timeout_obj,
    792     body=body,
    793     headers=headers,
```

```
    794     chunked=chunked,
    795     retries=retries,
    796     response_conn=response_conn,
    797     preload_content=preload_content,
    798     decode_content=decode_content,
    799     **response_kw,
    800 )
    802 # Everything went great!

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:536, in HTTPConnectionPool._make_request(self, conn␣
 ↪method, url, body, headers, retries, timeout, chunked, response_conn,␣
 ↪preload_content, decode_content, enforce_content_length)
    535 except (BaseSSLError, OSError) as e:
--> 536     self._raise_timeout(err=e, url=url, timeout_value=read_timeout)
    537     raise

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:367, in HTTPConnectionPool._raise_timeout(self, err␣
 ↪url, timeout_value)
    366 if isinstance(err, SocketTimeout):
--> 367     raise ReadTimeoutError(
    368         self, url, f"Read timed out. (read timeout={timeout_value})"
    369     ) from err
    371 # See the above comment about EAGAIN in Python 3.

ReadTimeoutError: HTTPSConnectionPool(host='nominatim.openstreetmap.org',␣
 ↪port=443): Read timed out. (read timeout=1)

The above exception was the direct cause of the following exception:

MaxRetryError                             Traceback (most recent call last)
File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪requests/adapters.py:644, in HTTPAdapter.send(self, request, stream, timeout,␣
 ↪verify, cert, proxies)
    643 try:
--> 644     resp = conn.urlopen(
    645         method=request.method,
    646         url=url,
    647         body=request.body,
    648         headers=request.headers,
    649         redirect=False,
    650         assert_same_host=False,
    651         preload_content=False,
    652         decode_content=False,
    653         retries=self.max_retries,
    654         timeout=timeout,
    655         chunked=chunked,
    656     )
```

```
    658 except (ProtocolError, OSError) as err:

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:871, in HTTPConnectionPool.urlopen(self, method,␣
 ↪url, body, headers, retries, redirect, assert_same_host, timeout,␣
 ↪pool_timeout, release_conn, chunked, body_pos, preload_content,␣
 ↪decode_content, **response_kw)
    868     log.warning(
    869         "Retrying (%r) after connection broken by '%r': %s", retries,␣
 ↪err, url
    870     )
--> 871     return self.urlopen(
    872         method,
    873         url,
    874         body,
    875         headers,
    876         retries,
    877         redirect,
    878         assert_same_host,
    879         timeout=timeout,
    880         pool_timeout=pool_timeout,
    881         release_conn=release_conn,
    882         chunked=chunked,
    883         body_pos=body_pos,
    884         preload_content=preload_content,
    885         decode_content=decode_content,
    886         **response_kw,
    887     )
    889 # Handle redirect?

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:871, in HTTPConnectionPool.urlopen(self, method,␣
 ↪url, body, headers, retries, redirect, assert_same_host, timeout,␣
 ↪pool_timeout, release_conn, chunked, body_pos, preload_content,␣
 ↪decode_content, **response_kw)
    868     log.warning(
    869         "Retrying (%r) after connection broken by '%r': %s", retries,␣
 ↪err, url
    870     )
--> 871     return self.urlopen(
    872         method,
    873         url,
    874         body,
    875         headers,
    876         retries,
    877         redirect,
    878         assert_same_host,
    879         timeout=timeout,
    880         pool_timeout=pool_timeout,
```

```
    881         release_conn=release_conn,
    882         chunked=chunked,
    883         body_pos=body_pos,
    884         preload_content=preload_content,
    885         decode_content=decode_content,
    886         **response_kw,
    887     )
    889 # Handle redirect?

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/connectionpool.py:841, in HTTPConnectionPool.urlopen(self, method,␣
 ↪url, body, headers, retries, redirect, assert_same_host, timeout,␣
 ↪pool_timeout, release_conn, chunked, body_pos, preload_content,␣
 ↪decode_content, **response_kw)
    839     new_e = ProtocolError("Connection aborted.", new_e)
--> 841 retries = retries.increment(
    842     method, url, error=new_e, _pool=self, _stacktrace=sys.exc_info()[2]
    843 )
    844 retries.sleep()

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪urllib3/util/retry.py:535, in Retry.increment(self, method, url, response,␣
 ↪error, _pool, _stacktrace)
    534     reason = error or ResponseError(cause)
--> 535     raise MaxRetryError(_pool, url, reason) from reason  # type:␣
 ↪ignore[arg-type]
    537 log.debug("Incremented Retry for (url='%s'): %r", url, new_retry)

MaxRetryError: HTTPSConnectionPool(host='nominatim.openstreetmap.org', port=443 :
 ↪ Max retries exceeded with url: /reverse?lat=39.6403&lon=-106.
 ↪3742&format=json&addressdetails=1 (Caused by␣
 ↪ReadTimeoutError("HTTPSConnectionPool(host='nominatim.openstreetmap.org',␣
 ↪port=443): Read timed out. (read timeout=1)"))

During handling of the above exception, another exception occurred:

ConnectionError                           Traceback (most recent call last)
File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/geopy
 ↪adapters.py:482, in RequestsAdapter._request(self, url, timeout, headers)
    481 try:
--> 482     resp = self.session.get(url, timeout=timeout, headers=headers)
    483 except Exception as error:

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
 ↪requests/sessions.py:602, in Session.get(self, url, **kwargs)
    601 kwargs.setdefault("allow_redirects", True)
--> 602 return self.request(     , url, **kwargs)
```

```
File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
↪requests/sessions.py:589, in Session.request(self, method, url, params, data,␣
↪headers, cookies, files, auth, timeout, allow_redirects, proxies, hooks,␣
↪stream, verify, cert, json)
    588 send_kwargs.update(settings)
--> 589 resp = self.send(prep, **send_kwargs)
    591 return resp

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
↪requests/sessions.py:703, in Session.send(self, request, **kwargs)
    702 # Send the request
--> 703 r = adapter.send(request, **kwargs)
    705 # Total elapsed time of the request (approximately)

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/
↪requests/adapters.py:677, in HTTPAdapter.send(self, request, stream, timeout,␣
↪verify, cert, proxies)
    675         raise SSLError(e, request=request)
--> 677     raise ConnectionError(e, request=request)
    679 except ClosedPoolError as e:

ConnectionError: HTTPSConnectionPool(host='nominatim.openstreetmap.org',␣
↪port=443): Max retries exceeded with url: /reverse?lat=39.6403&lon=-106.
↪3742&format=json&addressdetails=1 (Caused by␣
↪ReadTimeoutError("HTTPSConnectionPool(host='nominatim.openstreetmap.org',␣
↪port=443): Read timed out. (read timeout=1)"))

During handling of the above exception, another exception occurred:

GeocoderUnavailable                       Traceback (most recent call last)
Cell In[4], line 19
     16     # Add a delay to respect API rate limits (crucial for larger␣
↪dataframes)
     17     time.sleep(10)
---> 19 ski_resorts_df['State'] = ski_resorts_df.apply(get_state_name, axis=1)
     20 ski_resorts_df.head()

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/panda
↪core/frame.py:12419, in DataFrame.apply(self, func, axis, raw, result_type,␣
↪args, by_row, engine, engine_kwargs, **kwargs)
  12405         raise ValueError(f"Unknown engine '{engine}'")
  12407     op = frame_apply(
  12408         self,
  12409         func=func,
   (…)  12417         kwargs=kwargs,
  12418     )
> 12419     return op.apply().__finalize__(self, method="apply")
  12420 elif hasattr(engine, "__pandas_udf__"):
  12421     if result_type is not None:
```

```
File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/panda /
 ↪core/apply.py:1015, in FrameApply.apply(self)
   1012 elif self.raw:
   1013     return self.apply_raw(engine=self.engine, engine_kwargs=self.
 ↪engine_kwargs)
-> 1015 return self.apply_standard()

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/panda /
 ↪core/apply.py:1167, in FrameApply.apply_standard(self)
   1165 def apply_standard(self):
   1166     if self.engine == "python":
-> 1167         results, res_index = self.apply_series_generator()
   1168     else:
   1169         results, res_index = self.apply_series_numba()

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/panda /
 ↪core/apply.py:1183, in FrameApply.apply_series_generator(self)
   1180 results = {}
   1182 for i, v in enumerate(series_gen):
-> 1183     results[i] = self.func(v, *self.args, **self.kwargs)
   1184     if isinstance(results[i], ABCSeries):
   1185         # If we have a view on v, we need to make a copy because
   1186         #  series_generator will swap out the underlying data
   1187         results[i] = results[i].copy(deep=False)

Cell In[4], line 9, in get_state_name(row)
      7 coordinates = f"{row['Latitude']}, {row['Longitude']}"
      8 # Use geolocator.reverse() to get location data
----> 9 location = geolocator.reverse(coordinates, exactly_one=True)
     10 # Extract the state name from the raw address dictionary
     11 if location and 'address' in location.raw:
     12     # 'state' is the key for state name in the Nominatim response

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/geopy
 ↪geocoders/nominatim.py:372, in Nominatim.reverse(self, query, exactly_one,␣
 ↪timeout, language, addressdetails, zoom, namedetails)
    370 logger.debug("%s.reverse: %s", self.__class__.__name__, url)
    371 callback = partial(self._parse_json, exactly_one=exactly_one)
--> 372 return self._call_geocoder(url, callback, timeout=timeout)

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/geopy
 ↪geocoders/base.py:368, in Geocoder._call_geocoder(self, url, callback,␣
 ↪timeout, is_json, headers)
    366 try:
    367     if is_json:
--> 368         result =␣
 ↪self.adapter.get_json(url, timeout=timeout, headers=req_headers)
```

```
    369     else:
    370         result = self.adapter.get_text(url, timeout=timeout,␣
 ↪headers=req_headers)

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/geopy/
 ↪adapters.py:472, in RequestsAdapter.get_json(self, url, timeout, headers)
    471 def get_json(self, url, *, timeout, headers):
--> 472     resp = self._request(url, timeout=timeout, headers=headers)
    473     try:
    474         return resp.json()

File ~/Desktop/Projects/powderpipeline/.venv/lib/python3.12/site-packages/geopy/
 ↪adapters.py:494, in RequestsAdapter._request(self, url, timeout, headers)
    492         raise GeocoderServiceError(message)
    493     else:
--> 494         raise GeocoderUnavailable(message)
    495 elif isinstance(error, requests.Timeout):
    496     raise GeocoderTimedOut("Service timed out")

GeocoderUnavailable: HTTPSConnectionPool(host='nominatim.openstreetmap.org',␣
 ↪port=443): Max retries exceeded with url: /reverse?lat=39.6403&lon=-106.
 ↪3742&format=json&addressdetails=1 (Caused by␣
 ↪ReadTimeoutError("HTTPSConnectionPool(host='nominatim.openstreetmap.org',␣
 ↪port=443): Read timed out. (read timeout=1)"))
```

[ ]:
```
selected_states = ['Washington', 'Oregon', 'Idaho']
ski_resorts_df_local = ski_resorts_df[ ski_resorts_df['State'].
 ↪isin(selected_states) ]

base_map = alt.Chart(states).mark_geoshape(
    fill='lightgray',
    stroke='white'
).project(
    "albersUsa"
).properties(
    title='Ski Resorts Local to US',
    width=600,
    height=400
).transform_filter(
    alt.FieldOneOfPredicate(field='properties.name', oneOf=selected_states)
)

points = alt.Chart(ski_resorts_df_local).mark_circle().encode(
    longitude='Longitude:Q',
    latitude='Latitude:Q',
    color=alt.Color('Pass_Affiliation:N', scale=alt.Scale(scheme='viridis')), #␣
 ↪Color by the specified column
```

```
    size=alt.value(50), # Set a fixed size for better visibility
    tooltip=['Resort_Name', 'Pass_Affiliation'] # Add tooltips for interactivity
)

# Combine the base map and the points
chart = base_map + points
chart.save('../public/ski_resorts.png')

# Display the final chart
chart
```

[ ]: (76, 6)

4 Snowfall over the past Decade

Ok, let's take a look at snowfall over the past decade. This is all to determine the amount of snow at each resort, and therefore the number of days each resort will be open and then the number of "good" days.

There is no single "magic number" of inches required for a ski resort to open. The minimum amount of snow needed varies drastically depending on the mountain's terrain, the density of the snow, and the resort's snowmaking capabilities. Here is a general rule of thumb for how much compacted snow (the "base") is required[1]: Grassy, smooth slopes (or heavy snowmaking): 12 to 18 inches[2][3] Standard, mixed-terrain slopes: 18 to 24 inches[3][4] Rocky, steep, or off-piste terrain: 36 to 48+ inches (3 to 4 feet)[1][4] To give you an idea of how this looks across the country, here is a list of prominent U.S. ski resorts and the estimated minimum snow depth they need to start opening their lifts:

4.0.1 1. Arapahoe Basin (Colorado)

Minimum Snow Needed: ~18 to 20 inches[4] Why: Known for being one of the first resorts to open in the U.S. (often in October), "A-Basin" uses a mix of high-altitude natural snow and snowmaking. Because the early-season High Noon trail is relatively smooth and heavily prepped, they can open their first runs with a packed base of just under two feet[4]. ### 2. The Summit at Snoqualmie (Washington) Minimum Snow Needed: 18 to 24 inches (Summit West) vs. 48+ inches (Alpental)[4][5] Why: This resort is a perfect example of how terrain dictates snow requirements. The "Summit West" area consists of smooth, grassy slopes that have been brush-cut in the summer, meaning they only need about 1.5 to 2 feet of heavy Pacific Northwest snow to open[5]. However, their sister peak, Alpental, is incredibly steep and rocky, requiring at least 4 feet of snow before it is safe to ski[4][5]. ### 3. Killington Resort (Vermont) Minimum Snow Needed: 12 to 18 inches Why: Dubbed the "Beast of the East," Killington relies heavily on massive, state-of-the-art snowmaking systems to open as early as November. Because man-made snow is much denser and more durable than natural powdery snow, they can pack down a highly durable 12-to-18-inch base on their initial grassy slopes to get the lifts spinning early[2][3]. ### 4. Mountain High (California) Minimum Snow Needed: ~12 inches[4] Why: Located in Southern California, Mountain High relies heavily on man-made snow[3]. Their lower-mountain beginner and intermediate groomers are graded relatively smooth, allowing them to open early-season runs with as little as a foot of dense, heavily compacted machine-made snow[4]. ### 5. Mammoth Mountain (California) Minimum

Snow Needed: 20 to 24 inches (Lower Mountain) vs. 40 to 60+ inches (Upper Mountain)[4] Why: Mammoth can open its lower-elevation, heavily groomed trails with about two feet of snow[4]. However, the famous upper bowls are situated above the tree line and are notoriously rocky. To safely open the steep upper-mountain chutes without skiers destroying their equipment on jagged rocks, Mammoth often needs well over 4 to 5 feet of snow base. ### 6. Timberline Lodge / Mt. Hood (Oregon) Minimum Snow Needed: 24 to 36 inches (Groomers) vs. 8+ feet (Off-Piste)[4] Why: Located on a volcano, the terrain off the groomed trails is littered with massive boulders, crags, and tree stumps[4]. While the resort can open its heavily managed groomed runs with a few feet of snow, locals know that the off-piste freeride areas aren't truly safe to explore until the mountain receives 8 feet (96 inches) of base to entirely bury the volcanic rock[4].

4.1 Key Factors That Determine Opening Day

If you are waiting for your local mountain to open, keep these three things in mind: Snow Density: 20 inches of heavy, wet "Cascade Concrete" creates a much better base than 20 inches of light, dry "Champagne Powder." Grooming machines press the air out of dry snow, meaning 2 feet of fluffy powder might compress down to just 4 inches of actual base[5]. Man-Made vs. Natural: Resorts with strong snowmaking infrastructure need less natural snowfall, as man-made snow is basically tiny ice pellets that form an incredibly strong, durable foundation[2][3]. The "Sacrifice" Skis: When resorts open with the bare minimum (under 24 inches), the conditions are often called "early season conditions"[1][4]. This means there will likely be exposed rocks, dirt, and twigs, which is why veteran skiers use old, beat-up "rock skis" rather than brand-new equipment for the first few weeks of the season[6][7].